Lecture 26 — Bitmask DP

> Year Curriculum · Prerequisites: Week 9 (bit manipulation), Week 14–15 (DP I & II), Week 8 (backtracking) · Estimated time: 2–2.5 hours

1. Overview

When a problem asks you to order, assign or group a **small** set of items — "visit every city once", "give each worker one job", "split the people into elevator rides" — the brute force usually enumerates permutations, n! of them. For n = 20 that is 2.4·10^18: hopeless. But very often the future of a partial solution depends only on **which** items have been used so far (and maybe on one extra piece of information such as the last item), not on the order in which they were used. Then we can use the **set of used items** as a DP state. Encoding the set as an n-bit integer (a *bitmask*) gives 2^n states — about a million for n = 20.

This lecture covers four recurring patterns:

1. `dp[mask][last]` for path problems (TSP, Hamiltonian paths): O(2^n · n²).
2. `dp[mask]` with the "current index" implied by `popcount(mask)` (assignment, matching): O(2^n · n).
3. Pair-valued states for packing problems (Elevator Rides): O(2^n · n).
4. Submask enumeration for partition problems: O(3^n).

The constraint **n ≤ 20** (or ≤ 16, ≤ 18) in a statement is the strongest hint that one of these is intended.

2. Subsets as integers

Item `i` (0-indexed) is in the set `mask` iff bit `i` is 1. The operations from Week 9:

Operation	Code
test `i ∈ mask`	`(mask >> i) & 1`
add `i`	`mask | (1 << i)`
remove `i`	`mask & ~(1 << i)`
toggle `i`	`mask ^ (1 << i)`
size	`__builtin_popcount(mask)`
lowest element	`__builtin_ctz(mask)` (mask ≠ 0)
lowest element as a bit	`mask & -mask`

Operation	Code
full set	`(1 << n) - 1`

Two properties matter for DP order. Adding an element always **increases** the integer, and every subset of `mask` is numerically `≤ mask`. Hence iterating `mask` from 0 to 2^n − 1 visits every set after all of its subsets — a valid topological order for transitions that add elements.

3. Pattern 1 — dp[mask][last]: TSP and Hamiltonian paths

3.1 Travelling salesman (Held–Karp)

Given n cities and distances `d[i][j]`, find the shortest closed tour that starts at city 0, visits every city exactly once and returns to 0.

State. `dp[mask][v]` = minimum cost of a path that starts at 0, visits exactly the cities in `mask` (which contains 0 and v), and currently stands at `v`.

Why this state suffices. Consider two partial paths with the same visited set and the same endpoint, say `0→2→1→3` and `0→1→2→3`. Any way of completing one of them is also a valid completion of the other, at the same additional cost. So only the cheaper of the two matters, and we can forget the order. This collapses (n−1)! orderings into 2^n·n states.

Transition (push form). For each `u ∉ mask`: `dp[mask | 1<<u][u] = min(…, dp[mask][v] + d[v][u])`.

Answer. `min over v ≠ 0 of dp[full][v] + d[v][0]`.

Complexity. 2^n·n states with n transitions each: O(2^n·n²) time, O(2^n·n) memory. For n = 20 that is ~4·10^8 very cheap operations (about a second in C++ with the loop order shown) and 2·10^7 cells — 160 MB as `long long`, so check the memory limit and prefer `int` when values fit.

Worked example. With the 4×4 matrix

```
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

the optimum is the tour 0 → 1 → 3 → 2 → 0 with cost 10 + 25 + 30 + 15 = 80, which the reference program prints.

3.2 Counting Hamiltonian paths (CSES 1690)

Hamiltonian Flights: count routes from city 1 to city n that visit every city exactly once, over directed flights (n ≤ 20, m ≤ 400), modulo 10^9+7. The state is the same; we sum instead of minimizing. Two prunings halve the work: masks that do not contain city 1 are unreachable, and city n may appear only in the full mask (the route must end there). Using **incoming** adjacency lists in "pull" form, `dp[mask][u] = Σ dp[mask \ {u}][v]` over flights `v → u`, makes the total work O(2^n · m).

4. Pattern 2 — dp[mask] with an implicit index: assignment

n workers and n jobs, `cost[i][j]` for worker i doing job j; assign jobs bijectively at minimum total cost. If we assign workers in the fixed order 0, 1, 2, …, then after assigning `k` workers, exactly `k` jobs are used — so the next worker is `popcount(mask)`. We do not need to store it.

```
dp[0] = 0
dp[mask | 1<<j] = min(…, dp[mask] + cost[popcount(mask)][j])    for j ∉ mask
```

That is O(2^n·n) time and O(2^n) memory. (For large n the Hungarian algorithm solves assignment in O(n³) — World Finals track — but the bitmask DP is often easier to adapt to extra constraints.)

AtCoder DP Contest O *Matching* counts perfect matchings in a bipartite compatibility matrix with exactly this DP (n ≤ 21); see Section 7.3.

5. Pattern 3 — pair-valued states: CSES 1653 *Elevator Rides*

n ≤ 20 people with weights `w[i]`, elevator capacity x. Find the minimum number of rides.

A natural state "set of people already transported" is not enough by itself: two arrangements with the same number of rides differ in how full the current (last) ride is. But among arrangements of the same set, we prefer (1) fewer rides, then (2) a lighter last ride. So store the lexicographically smallest pair:

```
best[mask] = (rides, weight of the last ride)
```

To compute `best[mask]`, try every person `p ∈ mask` as the last one to board: start from `best[mask \ {p}] = (r, last)`; if `last + w[p] ≤ x`, p joins the current ride `(r, last + w[p])`; otherwise p starts a new ride `(r + 1, w[p])`. Take the minimum over p. Initialize `best[0] = (1, 0)` — one empty ride open.

Why is it correct? An exchange argument: any optimal packing can be produced by boarding people one at a time in ride order, and with the same set boarded, a state with fewer rides or (equal rides and) a lighter last ride can simulate any continuation of a worse state. This runs in O(2^n·n), much faster than the O(3^n) partition approach of Pattern 4.

6. Pattern 4 — submask enumeration and O(3^n)

Some problems split the set into groups, where each group's value depends on the whole group. Then the transition picks the group containing some element:

```
dp[mask] = best over non-empty sub ⊆ mask of  dp[mask \ sub] + value[sub]
```

6.1 Enumerating submasks

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { ... }
```

`sub - 1` turns the lowest set bit of `sub` into 0 and all lower bits into 1; `& mask` keeps only bits of `mask`. The result is the next smaller submask, so the loop lists all non-empty submasks in decreasing order (handle the empty submask separately if needed).

6.2 The 3^n bound

Over all masks the number of iterations is `Σ_mask 2^{|mask|} = Σ_k C(n,k)·2^k = 3^n` by the binomial theorem. Intuitively each element is independently "outside mask", "in mask but not in sub" or "in sub". 3^16 ≈ 4.3·10^7 is fast; 3^20 ≈ 3.5·10^9 is usually too slow.

6.3 Avoiding repeated partitions — AtCoder DP U *Grouping*

n ≤ 16 rabbits, compatibility `a[i][j]`; split them into groups, scoring Σ `a[i][j]` over pairs in the same group; maximize. Without care, the same partition {A, B} is generated as "A last" and "B last". Fix the group that contains the **lowest** element of `mask` (require `sub & lowbit(mask)`). Each partition is then built in exactly one way. This does not change the asymptotic bound but halves the constant and, for counting problems, is required for correctness.

6.4 Precomputing subset values

`score[mask]` for all masks in O(2^n·n): take the lowest element `low`, `rest = mask \ {low}`, then `score[mask] = score[rest] + Σ_{j ∈ rest} a[low][j]`. The same "reuse the mask without its lowest bit" idea computes subset sums in O(2^n).

7. Reference implementations

All programs were compiled with `g++ -std=c++17 -O2 -Wall` and checked on the official samples (TSP example → 80, Hamiltonian Flights sample → 2, Elevator Rides sample → 2, Matching sample → 3, Grouping samples → 20, 0, 4999999999).

7.1 TSP (Held–Karp)

```cpp
#include <bits/stdc++.h>
using namespace std;

// Held-Karp TSP: shortest Hamiltonian cycle starting and ending at city 0.
// Input: n, then an n x n distance matrix. O(2^n * n^2) time, O(2^n * n) memory.
int main() {
    int n;
    cin >> n;
    vector<vector<long long>> d(n, vector<long long>(n));
    for (auto& row : d) for (auto& x : row) cin >> x;
    const long long INF = LLONG_MAX / 4;
    // dp[mask][v] = min cost of a path that starts at 0, visits exactly `mask`, ends at
v
    vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;                    // every path contains city 0
        for (int v = 0; v < n; v++) {
            if (!(mask >> v & 1) || dp[mask][v] >= INF) continue;
            for (int u = 0; u < n; u++) {
                if (mask >> u & 1) continue;
                int nmask = mask | (1 << u);
                dp[nmask][u] = min(dp[nmask][u], dp[mask][v] + d[v][u]);
            }
        }
    }
    int full = (1 << n) - 1;
    long long best = (n == 1) ? 0 : INF;
    for (int v = 1; v < n; v++) best = min(best, dp[full][v] + d[v][0]);
    cout << best << '\n';
}
```

7.2 CSES 1690 *Hamiltonian Flights*

```cpp
#include <bits/stdc++.h>
using namespace std;

// CSES 1690 - Hamiltonian Flights: count routes 1 -> n visiting every city once.
const int MOD = 1'000'000'007;

int main() {
    ios::sync_with_stdio(false);
```

```
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> in(n);              // in[u] = list of v with a flight v -> u
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        in[b - 1].push_back(a - 1);
    }
    // dp[mask][v]: number of routes from city 0 visiting exactly `mask`, ending at v.
    vector<vector<int>> dp(1 << n, vector<int>(n, 0));
    dp[1][0] = 1;
    for (int mask = 1; mask < (1 << n); mask++) {
        if (!(mask & 1)) continue;                          // must contain the start
        if ((mask >> (n - 1) & 1) && mask != (1 << n) - 1) continue; // city n only at
the end
        for (int u = 1; u < n; u++) {
            if (!(mask >> u & 1)) continue;
            int prev = mask ^ (1 << u);
            long long ways = 0;
            for (int v : in[u])
                if (prev >> v & 1) ways += dp[prev][v];
            dp[mask][u] = ways % MOD;
        }
    }
    cout << dp[(1 << n) - 1][n - 1] << '\n';
}
```

7.3 AtCoder DP O *Matching*

```
#include <bits/stdc++.h>
using namespace std;

// AtCoder DP Contest O - Matching: count perfect matchings between n men and
// n women (compatible[i][j] = 1), mod 1e9+7. dp[mask] where man popcount(mask)
// is the next one to be matched. O(2^n * n).
const int MOD = 1'000'000'007;

int main() {
    int n;
    cin >> n;
    vector<vector<int>> ok(n, vector<int>(n));
    for (auto& r : ok) for (auto& v : r) cin >> v;
    vector<int> dp(1 << n, 0);
    dp[0] = 1;
    for (int mask = 0; mask < (1 << n); mask++) {
        if (!dp[mask]) continue;
        int i = __builtin_popcount(mask);          // next man to match
        if (i >= n) continue;
        for (int j = 0; j < n; j++)
            if (ok[i][j] && !(mask >> j & 1)) {
                int nm = mask | (1 << j);
                dp[nm] = (dp[nm] + dp[mask]) % MOD;
```

```
            }
        }
        cout << dp[(1 << n) - 1] << '\n';
}
```

7.4 CSES 1653 *Elevator Rides*

```
#include <bits/stdc++.h>
using namespace std;

// CSES 1653 - Elevator Rides: minimum number of rides, capacity x.
// best[mask] = (rides used, weight in the last ride), compared lexicographically.
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    long long x;
    cin >> n >> x;
    vector<long long> w(n);
    for (auto& v : w) cin >> v;
    vector<pair<int, long long>> best(1 << n, {INT_MAX, 0});
    best[0] = {1, 0};                       // one (empty) ride open
    for (int mask = 1; mask < (1 << n); mask++) {
        for (int p = 0; p < n; p++) {
            if (!(mask >> p & 1)) continue;
            auto [rides, last] = best[mask ^ (1 << p)];
            if (last + w[p] <= x) {
                last += w[p];               // person p joins the current ride
            } else {
                rides++;                    // start a new ride with person p
                last = w[p];
            }
            best[mask] = min(best[mask], make_pair(rides, last));
        }
    }
    cout << best[(1 << n) - 1].first << '\n';
}
```

7.5 AtCoder DP U *Grouping* (submask enumeration)

```
#include <bits/stdc++.h>
using namespace std;

// AtCoder DP Contest U - Grouping: partition rabbits into groups maximizing score.
// Submask enumeration: total work O(3^n).
int main() {
    int n;
    cin >> n;
    vector<vector<long long>> a(n, vector<long long>(n));
    for (auto& r : a) for (auto& v : r) cin >> v;
    int N = 1 << n;
```

```
    vector<long long> score(N, 0);          // score of putting `mask` in one group
    for (int mask = 1; mask < N; mask++) {
        int low = __builtin_ctz(mask);
        int rest = mask ^ (1 << low);
        score[mask] = score[rest];
        for (int j = 0; j < n; j++)
            if (rest >> j & 1) score[mask] += a[low][j];
    }
    vector<long long> dp(N, LLONG_MIN);
    dp[0] = 0;
    for (int mask = 1; mask < N; mask++) {
        int low = mask & -mask;             // fix the lowest element's group to avoid
repeats
        for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
            if (!(sub & low)) continue;
            dp[mask] = max(dp[mask], dp[mask ^ sub] + score[sub]);
        }
    }
    cout << dp[N - 1] << '\n';
}
```

8. Common mistakes

- **Shifts beyond 31 bits.** `1 << 35` is undefined for `int`; use `1LL << i` when n > 30 (rare in DP, common in meet-in-the-middle).
- **Precedence.** `mask >> i & 1 == 0` parses as `mask >> i & (1 == 0)`. Parenthesize: `((mask >> i) & 1) == 0`.
- **Memory blow-up.** `long long dp[1 << 20][20]` is 160 MB. Use `int`, drop impossible states (e.g., masks without the start node), or roll the dimension.
- **Wrong iteration order.** Push transitions must go from smaller to larger masks. Increasing numeric order is enough because adding a bit increases the number.
- **Double counting partitions.** Fix the group of the lowest element.
- **Empty submask.** `dp[mask] = dp[mask ^ 0] + value[0]` refers to itself; the loop `sub > 0` excludes it.
- **Forgetting the start/end conditions** (TSP must return to 0; Hamiltonian Flights must end at n and not visit n earlier).
- **Using 3^n when 2^n·n suffices** — Elevator Rides with submask enumeration is O(3^20), too slow.

9. Practice problems

Warm-up:

- AtCoder DP Contest O — *Matching* — https://atcoder.jp/contests/dp/tasks/dp_o
- CSES 1690 — *Hamiltonian Flights* — https://cses.fi/problemset/task/1690
- Codeforces 580D — *Kefa and Dishes* — https://codeforces.com/problemset/problem/580/D

Core:

- CSES 1653 — *Elevator Rides* — https://cses.fi/problemset/task/1653
- AtCoder DP Contest U — *Grouping* — https://atcoder.jp/contests/dp/tasks/dp_u
- Codeforces 16E — *Fish* — https://codeforces.com/problemset/problem/16/E
- Codeforces 8C — *Looking for Order* — https://codeforces.com/problemset/problem/8/C

Challenge:

- Codeforces 11D — *A Simple Task* — https://codeforces.com/problemset/problem/11/D
- Codeforces 1238E — *Keyboard Purchase* — https://codeforces.com/problemset/problem/1238/E
- CSES 2181 — *Counting Tilings* (broken-profile DP) — https://cses.fi/problemset/task/2181

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 10 "Bit manipulation" (sections on DP over subsets).
- CP-Algorithms, "Enumerating submasks of a bitmask" — https://cp-algorithms.com/algebra/all-submasks.html
- USACO Guide, "Bitmask DP" — https://usaco.guide/gold/dp-bitmasks
- M. Held and R. M. Karp, "A Dynamic Programming Approach to Sequencing Problems", *J. SIAM*, 1962.
- Looking ahead: SOS DP (sum over subsets in O(2^n·n)) and broken-profile DP are covered in the World Finals track; next week continues with interval DP and digit DP.